

Sentiment Analysis

Use the Python programming language to create and train a sentiment analysis model for the Twitter dataset.

Dataset: Twitter Sentiment Dataset

This dataset is a train and test CSV of tweets along with a sentiment score.

<https://docs.google.com/file/d/0B04GJPshIjmPRnZManQwWEdTZjg/edit>

Requirements:

1. Sentiment Classifier:

- Use your preferred word/sentence encoding method (you do not need to implement this from scratch) to train a classifier that takes as an input a sentence and returns the sentiment.
- You may use any method but be prepared to discuss why you chose that method and how it may compare to some other approaches.

2. REST API:

- Implement a RESTful API using a framework of your choice, that exposes endpoints for:
 - Evaluating a sentence to return its sentiment.
 - We use Django and Django Rest Framework (DRF) in Avoma, you can also use it or anything else is fine too
- Ensure your API adheres to RESTful principles.

Tasks:

1. Model Implementation:

- Implement the sentiment classifier with at least two core functions:
 - train: This function trains the model.
 - evaluate: This function takes an input sentence and returns the sentiment.
- These functions can be part of a class or standalone functions in an IPython notebook or script.

2. API Endpoints:

- Create RESTful API endpoints:
 - POST /evaluate: Endpoint to evaluate a sentence and return the sentiment. This should handle the request and response synchronously.

General Notes:

- **Model Performance:**
 - We do not expect a state-of-the-art classifier. We are more interested in your ability to explain and discuss your methods and decisions, in addition to writing clean and maintainable code for training and running inference on the model.
- **Code Quality:**
 - Ensure your code is well-documented and follows best practices for code readability and maintainability.
- **RESTful Principles:**
 - Ensure your API design follows RESTful principles, including proper use of HTTP methods and status codes.
 - Don't worry too much about loading time for the model/endpoint
 - Brownie points for a good performant solution though.
- **Questions:**
 - Please let us know if you have any questions - don't be afraid to ask!

Submission:

- Provide your solution in a GitHub repository or a zipped folder containing:
 - The code for the sentiment classifier.
 - The REST API project.
 - Instructions on how to set up and run your project, including any necessary environment configurations.

This assignment evaluates your skills in machine learning, REST API development, and understanding of RESTful principles. Good luck!